LAB SHEET 2 — NoSQL DATABASE DESIGN

Only executable codes from Lab Sheet 2 of the supplied PDF. Q8 is comparison/theory, so it has no code.

Q1. Student database in RDBMS and MongoDB

```
-- MySQL
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE Students (
   Student_ID INT PRIMARY KEY,
   Name VARCHAR(50),
   Course VARCHAR(30),
   Semester INT,
   Marks INT
);
INSERT INTO Students VALUES
(1,'Aman','BCA',3,82),
(2,'Riya','BCA',3,91),
(3,'Karan','BCA',4,76),
(4,'Neha','BCA',4,88),
(5,'Rahul','BCA',3,69);

-- MongoDB (mongosh)
use CollegeDB
db.Students.insertMany([
 {Student_ID:1,Name:'Aman',Course:'BCA',Semester:3,Marks:82},
 {Student_ID:2,Name:'Riya',Course:'BCA',Semester:3,Marks:91},
 {Student_ID:3,Name:'Karan',Course:'BCA',Semester:4,Marks:76},
 {Student_ID:4,Name:'Neha',Course:'BCA',Semester:4,Marks:88},
 {Student_ID:5,Name:'Rahul',Course:'BCA',Semester:3,Marks:69}
])
db.Students.find()
```

Q2. MongoDB CRUD operations

```
use CollegeDB
db.Students.insertMany([
 {Student_ID:101,Name:'Aman',Course:'BCA',Semester:3,Marks:82},
 {Student_ID:102,Name:'Riya',Course:'BCA',Semester:3,Marks:91},
 {Student_ID:103,Name:'Karan',Course:'BSc',Semester:4,Marks:76},
 {Student_ID:104,Name:'Neha',Course:'BCA',Semester:4,Marks:88},
 {Student_ID:105,Name:'Rahul',Course:'BCom',Semester:2,Marks:69}
])
db.Students.find()
db.Students.find({Course:'BCA'})
db.Students.updateOne({Student_ID:101},{$set:{Marks:90}})
db.Students.deleteOne({Student_ID:105})
db.Students.countDocuments({})
```

Q3. MongoDB Orders with embedded documents

```
use CollegeDB
db.Orders.insertMany([
 {_id:1,customer:{id:101,name:'Aman'},products:[{name:'Mouse',qty:2,price:400},{name:'Keyboard',qty:1,price:800}]},
 {_id:2,customer:{id:102,name:'Riya'},products:[{name:'Monitor',qty:1,price:7000}]},
 {_id:3,customer:{id:101,name:'Aman'},products:[{name:'USB',qty:3,price:250}]},
 {_id:4,customer:{id:103,name:'Neha'},products:[{name:'Laptop',qty:1,price:45000}]},
 {_id:5,customer:{id:104,name:'Karan'},products:[{name:'Mouse',qty:1,price:400}]}
])
db.Orders.find({'customer.id':101})
db.Orders.updateOne({_id:1,'products.name':'Mouse'},{$set:{'products.$.qty':3}})
db.Orders.aggregate([
 {$unwind:'$products'},
 {$group:{_id:'$_id',total:{$sum:{$multiply:['$products.qty','$products.price']}}}}
])
db.Orders.aggregate([
 {$unwind:'$products'},
 {$group:{_id:'$_id',total:{$sum:{$multiply:['$products.qty','$products.price']}}}},
 {$match:{total:{$gt:1000}}}
])
```

Q4. Redis key-value database

```
# Redis CLI
SET student:1 "Aman"
SET student:2 "Riya"
SET student:3 "Karan"
SET student:4 "Neha"
```

```
SET student:5 "Rahul"

GET student:2
SET student:2 "Riya Sharma"
GET student:2
DEL student:5
KEYS student:*
```

Q5. Cassandra keyspace and Students table

```
CREATE KEYSPACE CollegeDB
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE CollegeDB;

CREATE TABLE Students (
 Student_ID int PRIMARY KEY,
 Name text, Course text, Semester int, Marks int
);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (1,'Aman','BCA',3,82);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (2,'Riya','BCA',3,91);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (3,'Karan','BCA',4,76);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (4,'Neha','BCA',4,88);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (5,'Rahul','BCom',2,69);

SELECT * FROM Students;
-- For course lookup, create a query-friendly table:
CREATE TABLE StudentsByCourse (
 Course text, Student_ID int, Name text, Semester int, Marks int,
 PRIMARY KEY (Course, Student_ID)
);
SELECT * FROM StudentsByCourse WHERE Course='BCA';
UPDATE Students SET Marks=95 WHERE Student_ID=2;
DELETE FROM Students WHERE Student_ID=5;
```

Q6. HBase Students table

```
# HBase shell
create 'Students', 'personal', 'academic'
put 'Students','101','personal:name','Aman'
put 'Students','101','personal:city','Roorkee'
put 'Students','101','academic:course','BCA'
put 'Students','101','academic:marks','82'
put 'Students','102','personal:name','Riya'
put 'Students','102','academic:course','BCA'
put 'Students','102','academic:marks','91'
get 'Students','101'
scan 'Students'
put 'Students','101','academic:marks','90'
delete 'Students','102','personal:name'
deleteall 'Students','102'
```

Q7. Student–Course graph in Neo4j

```
// Neo4j Browser / Cypher
CREATE (:Student {student_id:1,name:'Aman'});
CREATE (:Student {student_id:2,name:'Riya'});
CREATE (:Student {student_id:3,name:'Karan'});
CREATE (:Student {student_id:4,name:'Neha'});
CREATE (:Student {student_id:5,name:'Rahul'});
CREATE (:Course {course_id:'C1',name:'DBMS'});
CREATE (:Course {course_id:'C2',name:'Python'});
CREATE (:Course {course_id:'C3',name:'Java'});

MATCH (s:Student {student_id:1}),(c:Course {course_id:'C1'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:2}),(c:Course {course_id:'C2'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:3}),(c:Course {course_id:'C1'})
CREATE (s)-[:ENROLLED_IN]->(c);

MATCH (s:Student) RETURN s;
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {name:'DBMS'}) RETURN s;
MATCH (s:Student {student_id:1})-[:ENROLLED_IN]->(c) RETURN c;
MATCH (s:Student {student_id:4}),(c:Course {course_id:'C3'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:4})-[r:ENROLLED_IN]->(c)
DELETE r;
```